On the Security of SSH Client Signatures

Fabian Bäumer
Ruhr University Bochum
Bochum, Germany
fabian.baeumer@rub.de

Marcus Brinkmann
Ruhr University Bochum
Bochum, Germany
marcus.brinkmann@rub.de

Maximilian Radoy
University Paderborn
Paderborn, Germany
maximilian.radoy@upb.de

Jörg Schwenk
Ruhr University Bochum
Bochum, Germany
joerg.schwenk@rub.de

Juraj Somorovsky
University Paderborn
Paderborn, Germany
juraj.somorovsky@upb.de

Abstract

Administrators and developers use SSH client keys and signatures for authentication, for example, to access internet backbone servers or to commit new code on platforms like GitHub. However, unlike *servers*, SSH *clients* cannot be measured through internet scans. We close this gap in two steps. First, we collect *SSH client public keys*. Such keys are regularly published by their owners on open development platforms like GitHub and GitLab. We systematize previous non-academic work by subjecting these keys to various security tests in a longitudinal study. Second, in a series of black-box lab experiments, we analyze the implementations of algorithms for *SSH client signatures* in 24 popular SSH clients for Linux, Windows, and macOS.

We extracted 31,622,338 keys from three public sources in two scans. Compared to previous work, we see a clear tendency to abandon RSA signatures in favor of EdDSA signatures. Still, in January 2025, we found 98 broken short keys, 139 keys generated from weak randomness, and 149 keys with common or small factors—the large majority of the retrieved keys exposed no weakness.

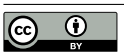
Weak randomness can not only compromise a secret key through its public key, but also through signatures. It is well-known that a bias in *random nonces* in ECDSA can reveal the secret key through public signatures. For the first time, we show that the use of *deterministic nonces* in ECDSA can also be dangerous: The private signing key of a PuTTY client can be recovered from just 58 valid signatures if ECDSA with NIST curve P-521 is used. PuTTY acknowledged our finding in CVE-2024-31497, and they subsequently replaced the nonce generation algorithm.

CCS Concepts

• **General and reference** → **Measurement**; • **Networks** → **Transport protocols**; • **Security and privacy** → **Digital signatures**; *Cryptanalysis and other attacks*.

Keywords

SSH, Authentication, Digital Signatures, Cryptographic Attacks, Measurement, Public Key Cryptography



This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '25, Taipei, Taiwan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1525-9/2025/10
https://doi.org/10.1145/3719027.3765079

ACM Reference Format:
Fabian Bäumer, Marcus Brinkmann, Maximilian Radoy, Jörg Schwenk, and Juraj Somorovsky. 2025. On the Security of SSH Client Signatures. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), October 13–17, 2025, Taipei, Taiwan.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3719027.3765079

1 Introduction

System administrators often use the Secure Shell Protocol (SSH) to deploy and manage servers on a private network or the public internet. It also secures other network connections, such as access to Git repositories containing the source code of public or private software projects. Thus, the security of SSH, and especially the security of SSH client authentication, is of critical importance. The IETF standardizes the basic SSH protocol [40–44], but includes a flexible extension mechanism. Over the years, vendors have added new algorithms, features, and countermeasures to known attacks. So, to understand the real security of SSH, it is not sufficient to analyze standard documents; an investigation of actual implementations and configurations is necessary. While the cryptographic capabilities of clients and servers can be investigated in the lab, the actual *configuration* of SSH clients must be measured in the real world. For servers, this can be done through internet scans (e.g., [29, 35, 58]).

SSH Clients. To measure the configuration of SSH clients, we need a strategy that is different from that for servers. Previous work used honeypots [38, 45, 63] and observed clients connecting to them via SSH. However, such data is heavily biased toward malicious clients, while we are interested in benign users. We note that SSH clients can authenticate using passwords or digital signatures. While passwords are only sent over an encrypted SSH connection, digital signatures rely on public keys that may be available for analysis. Thus, in our work, we propose to measure SSH clients that use signature-based authentication, which consists of two critical steps: secret key generation and signature generation.

Large-Scale Analysis of SSH Public Keys. If a secret key is improperly generated, its security can be compromised, given only the public key. For instance, the security of RSA relies on the difficulty of the integer factorization problem. However, if RSA keys are generated incorrectly, different moduli can share a common factor. This opens the door to a batch GCD attack, which can easily factor such vulnerable keys [12, 33]. Similar risks exist with (EC)DSA, where

flaws in key generation can lead to severe security vulnerabilities [11]. To perform a large-scale analysis of SSH keys, we retrieve SSH public keys published by their owners on GitHub, GitLab, and Launchpad. We then evaluate the distribution of signature algorithms among these keys and test the keys for known weaknesses, such as RSA keys with small factors or a vulnerability to ROCA (Return of Coppersmith's Attack) [48]. Compared to earlier work analyzing public keys from GitHub and GitLab [4, 6, 15, 22, 24] (see Section 4.1), we consider a larger set of keys from more diverse sources, and apply all known, efficient security tests against these data sources at two points in time.

Lab-Evaluation of SSH Client Signatures. The client key is used in authentication to sign, among other data, the username and a *session identifier* unique to the connection (Section 2.2). This signature is sent to the server over an encrypted channel and cannot be observed publicly. Thus, we analyze client signatures in the lab. SSH supports RSA, DSA, ECDSA, and EdDSA. The DSA and ECDSA algorithms require a nonce for each signature. Breitner and Heninger [20] have shown that weak randomness for nonce generation can lead to a secret key recovery attack, requiring only a small number of signatures for the attacker. To avoid the attack, they recommend the use of deterministic nonces, such as described in RFC 6979 [57] and recently included in FIPS 186-5 [50]. Using a novel black-box evaluation method, we analyze the (undocumented) methods for (EC)DSA nonce generation in SSH clients. To differentiate between random and deterministic nonces, we have to generate two signatures over the exact same payload, which is impossible in multiple SSH connections due to the use of unique client and server randoms in the session identifier. To overcome this, we let the client connect to a custom server, which triggers a standard-conforming partial authentication success message, provoking the client to authenticate multiple times using the same key over the same connection. This allows us to study the nonce generation algorithm of arbitrary SSH clients and agents over just a network connection (see Section 3.4).

In our evaluation, we found that PuTTY uses a non-standard deterministic nonce scheme, named k_{proto}. We found that this algorithm outputs highly biased nonces (the nine most significant bits are always zero) when used with the elliptic curve NIST P-521 [23]. By applying a solution to the associated hidden number problem, we can compute the private key with a success probability ≥ 0.5 from just 58 signatures, and always from 60 signatures.

Dangers of SSH Agents. Sometimes, SSH connections need to traverse firewalls and overcome network boundaries. In this case, public-facing SSH servers ("jump servers") can be used as intermediaries, acting as a proxy between the client and the server. There are two options: The proxy server may establish TCP port forwarding to allow for a direct connection between the client and the server. However, port forwarding can also be used to exfiltrate data from an organization's network, and is thus often disabled. Second, the client can use an SSH session to the proxy server and establish a second SSH connection between the proxy and target server from there (Figure 1). In this case, the proxy must be able to authenticate itself to the server *as the client*. With signature-based client authentication, this means that the *proxy must be able to sign* the authentication request with *the client's private key*. To enable the proxy to do so, the remote SSH client running on the proxy server can forward the signing request to a local signing oracle, the *SSH agent*. The SSH agent protocol [46] provides a high-bandwidth signing oracle for arbitrary messages. Thus, users are warned (for example, in the GitHub documentation [30]) to enable SSH agent forwarding only if they trust the proxy server. However, our analysis revealed two SSH implementations (AbsoluteTelnet and Tectia SSH) that enable SSH agent forwarding by default, ultimately giving all SSH servers that the user connects to the ability to sign arbitrary messages with the user's private key, if not explicitly disabled.

Contributions. We make the following contributions:

- We provide the first longitudinal analysis of SSH client signature keys in the wild, retrieved from Git-based repository hosting services (Section 4.1). We apply a thorough sequence of security tests on the collected keys, revealing 98 broken short keys, 139 keys generated from weak randomness, and 149 keys with common or small factors (Section 4.2).
- We evaluate the public key upload restrictions of these hosting services, which can protect users from uploading vulnerable keys to their accounts. We show that all services allow uploading some vulnerable keys, such as short RSA keys, ECDSA keys with points at infinity, or DSA keys with p or q not prime, among others (Section 4.3).
- We analyze signature generation algorithms implemented in SSH clients and agents. In particular, we present a novel method to measure an SSH client's (EC)DSA nonce generation algorithm over the network (Section 5.1).
- We demonstrate a novel attack on PuTTY's deterministic nonces for ECDSA, recovering a NIST P-521 private key from only 58 signatures (Section 5.2).

Artifacts. We share our toolset used in this research paper on Zenodo for permanent access. This toolset includes the SSH client key scraper, evaluation scripts for data aggregation, a script to generate test keys for testing public key upload restrictions, our optimized batch GCD implementation, the source code of the SSH signature sampling and analysis tool, a benchmarking script for the PuTTY P-521 private key recovery attack presented in Section 5.2, and additional material. Due to ethical concerns, we do not plan to make the database with all scraped public keys publicly available. The artifacts are available at:

https://doi.org/10.5281/zenodo.17008443

Ethical Considerations. We downloaded the public key data from the platforms through their official APIs, adhering to their rate limits. We obtained several factorizations for RSA moduli as a result of our checks on weak keys. To reduce the risk of a potential leak before the platform can revoke the affected keys, we performed the public key collection and analysis on a separate machine with limited access. We did not use these keys in a context other than the described vulnerability analysis; we did not recover or use any private keys to authenticate to any platform.

We disclosed our findings to the platforms, allowing them to inform the affected users, revoke vulnerable keys, and implement upload restrictions as necessary. GitLab responded positively, mitigated the report internally without providing further details, and currently plans to show a warning to users when they upload DSA

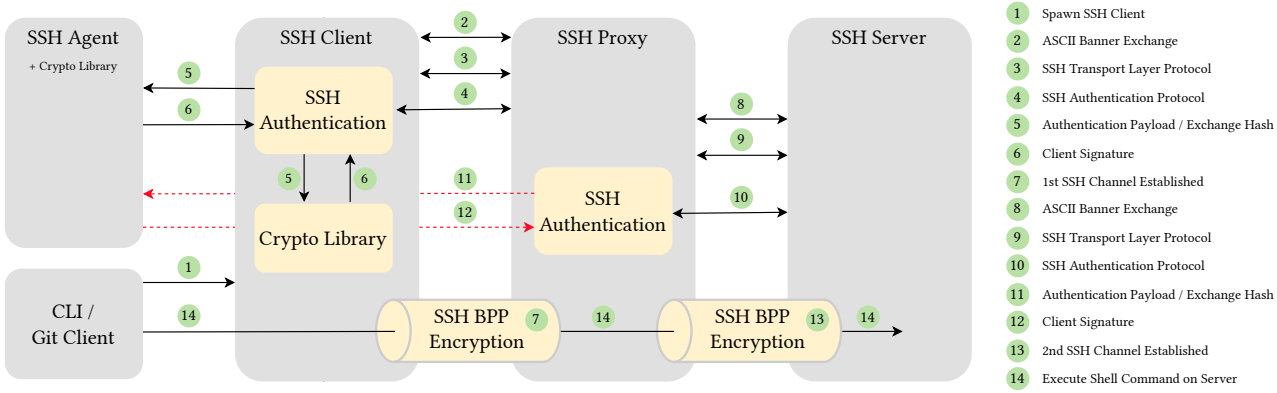


Figure 1: The SSH client ecosystem. (1) The user connects an SSH client to a proxy server, intending to connect to a server only accessible through the proxy. (2-4,7) The SSH protocol runs between the client and the proxy. (5-6) The signature for public key authentication is created either by the client directly or by an SSH agent. (8-10,13) A second SSH protocol runs between the proxy and the intended server. (11-12) The signature is created in the SSH agent local to the user. (14) The user can run shell commands through the two connected channels. As we will show later in Section 5.1, agent forwarding (11-12) is supported by many SSH clients, two of which enable it even by default and provide a signing oracle to SSH proxies.

or weak RSA keys, starting with release 18.3.[1] GitHub channels all vulnerability reports to a bug bounty program on HackerOne, whose policy states that users are self-responsible for the content in their repositories, which, to an extent, also applies to SSH public keys. A report that we opened, despite the bug bounties' policy, was closed as out of scope. We conclude that GitHub does not plan to take any further action or refine its upload restrictions. After initial response and disclosure, we did not receive any further response from Canonical regarding public keys on Launchpad.

Although the public keys we analyzed are publicly available, we currently do not plan to publish the complete datasets. The raw datasets contain millions of public keys alongside metadata, including usernames for all existing accounts on each platform and their respective account creation date. This, among other applications, could enable account linking and activity tracking across the three platforms based on the user's public key. While we do provide artifacts that can be used to create similar datasets, we deem the resources necessary to do so significant enough to justify our decision not to publish the raw datasets, thereby thwarting low-effort attacks. Additionally, not publishing the raw datasets maintains the platform operator's ability to restrict access to a user's SSH public keys in the future. The immediate evaluation results contain sensitive information, including partial and complete key factorizations of not yet revoked public keys.

We disclosed our attack on PuTTY (CVE-2024-31497) in early 2024. The PuTTY maintainer published a fix in PuTTY 0.81. We notified AbsoluteTelnet and Tectia SSH about the default setting for agent forwarding—AbsoluteTelnet intends to change the default behavior in the future.

[1]https://gitlab.com/gitlab-org/gitlab/-/issues/432624, accessed: 2025-07-18

2 Background

2.1 Attacks on Digital Signature Schemes

RSA. A direct way to attack RSA signatures is to compute factors of the RSA modulus, since this would allow attackers to calculate the private key. Notably, when two keys share a prime factor, factoring becomes as easy as calculating the greatest common divisor (GCD) of both moduli; for large collections of RSA moduli, batch GCD algorithms [12] can be applied. If a weak randomness source is used, this results in a higher probability that a prime factor is reused [65]. RSA is susceptible to *fault attacks* when employing the Chinese Remainder Theorem for modular exponentiation [5, 16]; such attacks have recently been applied to SSH server authentication [59]. If proprietary optimizations are used in key generation, this may also result in vulnerabilities [48]. Our complete toolchain to analyze RSA public keys is detailed in Section 3.2.

(EC)DSA. DSA and ECDSA are parts of the *Digital Signature Standard* (DSS, [49, 50]). DSA uses a subgroup G of the multiplicative group of a finite field $GF(p)$ whose order q is tailored to specific hash functions. To compute a DSA signature (r, s), the value s is computed from the hash value $h(m)$ of the message to be signed, the private key a of Alice, and $r = g^k$ computed from a random nonce k. It is known that using biased or repeated nonces k enables recovering the private key of Alice from a set of valid DSA signatures [20]. To mitigate this threat, the use of deterministic, pseudorandom nonces generated from a high-entropy secret value and some deterministic, non-repeating parameters was proposed [57].

In ECDSA [49], G is an elliptic curve of order q. In Appendix D to [49], NIST recommends a list of elliptic curves for ECDSA. While the bit lengths of the elliptic curve moduli P-192, P-224, P-256, and P-384 align with byte boundaries and hash lengths from the SHA-2 and SHA-3 standards, this is not the case with P-521—there is no typo switching the last two digits, but $p = 2^{521} - 1$ is one

of the extremely rare *Mersenne* prime numbers. This special form allows computations in $GF(2^{521}-1)$, and thus in the corresponding elliptic curve group, to be carried out very efficiently. Like their DSA counterparts, ECDSA algorithms allow the private key recovery if the nonces k are biased or reused.

2.2 SSH

Standards and Deployment. IETF standards [41, 42, 44] define the basic structure of SSH. There has been no major or minor version update since 2006, despite SSH's constant evolution and the addition of new cryptographic algorithms to its implementations. However, recently, an IETF working group tasked with formally documenting previously undocumented SSH features was established. In contrast to TLS, SSH is mainly deployed in small or closed ecosystems, where a small number of system administrators manage a small number of servers. Large platforms like GitHub offer their key registration services. As a result, no PKI or certificate support is necessary: Both clients and servers store lists of trusted credentials, typically public signature keys or username/password pairs.

SSH Protocol Architecture. Academic publications typically investigate authenticated key exchange (AKE) protocols, which establish authenticated symmetric keys for client and server, and binary encryption layers, which use these keys for authenticated encryption. The SSH RFCs [41, 42, 44] use a different terminology:

- The **Transport Layer Protocol** [44] consists of a handshake protocol, where only the server is authenticated, and an authenticated encryption layer called *Binary Packet Protocol (BPP)*. During the handshake, the server and client establish session keys for encryption, and the server authenticates itself to the client. To conclude the transport layer protocol, the client typically requests the *authentication protocol* as a subsequent service from the server.
- The **Authentication Protocol** [41] offers different methods to authenticate SSH clients and runs within the protected BPP layer of the Transport Layer Protocol. The primary justification for delaying client authentication after the session establishment is to allow the client to authenticate using a password instead of a public key signature. After successful execution, the client has authenticated to the server and specified a service name for the following message exchange. In virtually all cases, this is the connection protocol.
- The **Connection Protocol** [42] multiplexes logical channels over the encrypted tunnel. Each channel may be used to implement a single application layer feature, such as remote shells, X11 forwarding, or port forwarding. The protocol runs indefinitely until the connection is closed on either end.

In this paper, we focus on the *SSH Authentication Protocol*. We investigate deployed keys, supported signing algorithms, and the signature creation architecture.

SSH Handshake (Figure 2). To initiate an SSH connection, both peers exchange a version banner. The Binary Packet Protocol (BPP) is used from the third message onward, initially without encryption and authentication. In the `KEXINIT` messages, nonces and ordered lists of algorithms are exchanged: There is one list for key exchange, one for server signatures, and two (one per direction) each

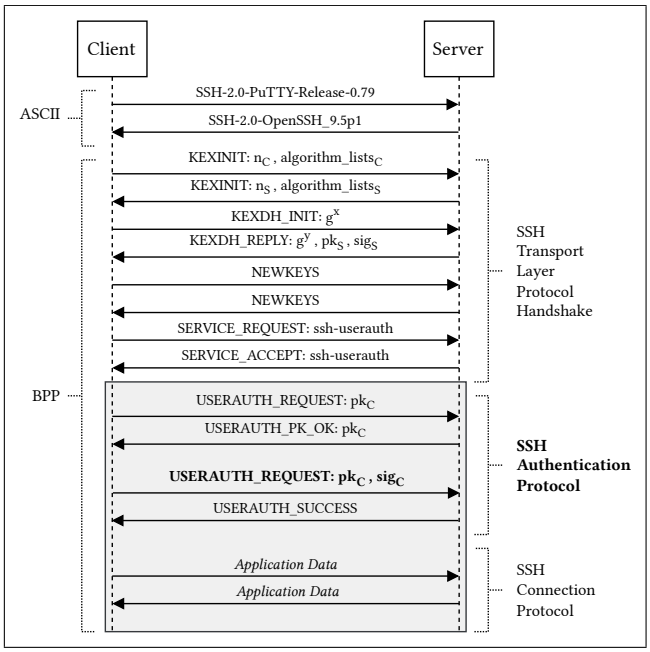


Figure 2: Typical SSH handshake using a finite-field Diffie-Hellman key exchange. In this paper, we investigate the security of SSH Authentication Protocol implementations, especially the deployed keys and the signature algorithms.

for encryption, MAC, and compression. For each list, the negotiated algorithm is the first algorithm in the client's list, which is also offered by the server. In the `KEXDH_INIT` and `KEXDH_REPLY` messages, a finite-field Diffie-Hellman key exchange is performed. SSH also supports elliptic curves (ECDH) and hybrid schemes with post-quantum cryptography (PQC) as alternatives. The server authenticates itself with a digital signature as part of the handshake. The signature is computed over the *session identifier*, a hash over the relevant parts of the key exchange.

2.3 Authentication Protocol Ecosystem

The SSH authentication protocol ecosystem is complex and distributed (Figure 1). This section explains its different components, which will be analyzed in Section 5.

SSH Authentication Protocol. To authenticate itself after the handshake, the client initiates the SSH Authentication Protocol [41] by proposing an authentication method, its username, and other parameters (e.g., password or public key/digital signature). SSH supports a variety of different authentication methods, among these:

- `password` – The client provides the user's password directly to the server for verification.
- `publickey` – The client sends a public key, and a signature over the session identifier and authentication request. Supported signature algorithms are based on RSA, DSA, ECDSA, and EdDSA.
- `hostbased` – Similar to `publickey`, but the public key is bound to the client's host rather than the user.

Table 1: Data to be signed by the client to perform signature-based client authentication as part of the SSH authentication protocol [41, Section 7]. Data types as per [43, Section 5].

Data Type	Value
`string`	session identifier
`byte`	SSH_MSG_USERAUTH_REQUEST
`string`	user name
`string`	service name
`string`	"publickey"
`boolean`	TRUE
`string`	public key algorithm name
`string`	public key to be used for authentication

- `keyboard-interactive` – The server may send one or more authentication prompts to the client, which are to be displayed and answered by the user.

If the server does not support the proposed authentication method, it may reject and respond with a list of supported methods. The client may request this list directly by proposing the authentication method `none`.

SSH Clients. If the authentication option `publickey` is chosen, the SSH client authenticates itself via a signature over the exchange hash (similar to the server) plus additional data. In detail, the client signs the data shown in Table 1. Clients may be configured with multiple public keys using different algorithms to connect to more than one server. Many proprietary implementations of SSH clients exist; for our study, we selected a variety of Linux, Windows, and macOS-based SSH clients (Table 4).

SSH Servers. The SSH server is authenticated to the client during the Transport Layer Protocol as part of the handshake. The server's SSH configuration files determine trusted public keys for signature-based client authentication and control which SSH connection protocol features are enabled for each session. For example, SSH access can be limited to a single shell command, or port forwarding can be disabled. The server can advertise the available public key signature algorithms for client authentication in an extension info message [14] sent as the first message on the encrypted session before the authentication protocol starts.

SSH Proxies. Proxies can be used in either of two configurations. As a *jump server*, they allow an authenticated client to access an SSH server on the remote network using port forwarding. In this case, a second SSH connection to a server within an organization's network is tunneled through the first SSH connection to the jump server, which the administrator of the jump server must enable.

Alternatively, the proxy server can act as both an SSH client and server, and thus as a legitimate man-in-the-middle between the SSH client and the destination server. In this case, there are two separate BPP channels: One secure channel between client and proxy (Figure 1, 7) and one between proxy and server (Figure 1, 13). In the second channel, the proxy (legitimately) can impersonate the client despite public key authentication by making use of *agent forwarding* in the first channel. Agent forwarding provides a signing oracle to the proxy, allowing it to sign the client's authentication request (Figure 1, 11) in the second channel without requiring the client to copy its private key files to the proxy.

SSH Agents. SSH clients may generate digital signatures or delegate signature generation to another program, a so-called *SSH agent*. When used, the SSH agent controls the client's private signing key. The *exchange hash* is computed by the SSH client and then handed over as part of the authentication payload to the SSH agent for signing. Two use cases for SSH agents are mentioned in the literature: (1) to protect the private key of the client by storing it only in the main memory of a separate process, and (2) to enable SSH proxies. SSH agents provide an interface with key management and signing functions [46]. The primary operations are loading and unloading signing keys, listing the available signing keys, and signing arbitrary data. SSH agents may implement support for constraints on stored keys (for example, requiring explicit user confirmation at the creation of each signature); these constraints require additional configuration. Although agents may limit remote signing to valid user authentication data structures, none of the tested agents enforced any check on the data to be signed—they act as *unconstrained, fully chosen message signing oracles*. SSH agents are popular because they give users access to a passphrase-protected private key without requiring them to enter the passphrase at each login.

Example. In Figure 1, a complex scenario for SSH client authentication is depicted. (1) By some shell/CLI command, an SSH connection to the SSH server is initiated. Since Git can be invoked through a CLI or a GUI that transforms the inputs into CLI commands, our Git use case is covered here. (2) The SSH client will now exchange ASCII banners; since an SSH proxy is configured here, this exchange is between the client and the proxy. (3) The SSH Transport Layer Protocol negotiates algorithms and keys and authenticates the proxy. (4,5,6) In the SSH Authentication Protocol, the client needs a signature over the authentication payload, including the exchange hash. If no proxy is involved, the client has two options: Use its crypto library with its private key or invoke the SSH agent. After establishing the first secure channel (7), the SSH proxy, acting as an SSH client, initiates another banner exchange, this time with the SSH server (8). After the server is authenticated and the BPP parameters are negotiated (9), the SSH proxy can (legitimately) impersonate the client. (11) The authentication payload, including the exchange hash of the second SSH session, is sent to the SSH agent through the first secure BPP channel (7). This is possible because the client has enabled agent forwarding. The SSH proxy receives the signature (12) and completes the protocol (10), establishing a second secure BPP channel (13) between the proxy and server. (14) CLI/shell commands can now be executed on the server securely.

3 Methodology

Our methodology is designed to assess the security of SSH client signatures in the wild. As actual client signatures can only be observed by trusted servers, they can not be collected directly. We therefore approach this issue in multiple steps. First, we collect a large number of SSH public keys from Git-based online collaborative development platforms, namely GitHub, GitLab, and Launchpad, as described in Section 3.1. The fact that these platforms reveal

the SSH public keys of their users has previously been noted by engineers [22, 31, 62], who highlighted the risk to user's privacy.[2] We then subject these keys to an extensive test suite based on the rules of the CA/Browser forum for TLS server certificates [28], detailed in Section 3.2. Compared to previous work, which limits its scope to specific vulnerabilities or key types, we provide the first comprehensive analysis of all SSH public keys on these platforms. In Section 3.3, we analyze these platforms for upload restrictions that are designed as a protection mechanism to prevent users from uploading weak or compromised keys.

Following the analysis of public keys, we also conduct extensive tests of SSH clients and agents in a controlled environment, as described in Section 3.4. In particular, we focus on identifying the underlying cryptographic library and supported algorithms for each analyzed SSH client, followed by checks for deterministic or biased nonces.

3.1 Large-Scale Collection of SSH Client Keys

For our analysis in Section 4, we collect publicly accessible SSH client keys from the online collaborative development platforms GitHub, GitLab, and Launchpad. These platforms commonly use SSH keys to authenticate users accessing source code repositories for reading and writing. We exclude Bitbucket from our study, although it is the third-largest platform, because downloading user SSH keys requires authorized access to workspaces. For each platform, keys are collected through the official GraphQL or REST APIs, respecting the platform's API limits. We use the GraphQL API for GitHub, and a combined approach for GitLab, using GraphQL to list users and REST to retrieve their SSH public keys. Launchpad only offers a REST API.

Each API response is stored in an Elasticsearch database for further processing. As API responses typically list SSH public keys by user, entries are transposed to list users by SSH public key instead. The resulting entries are checked for duplicate keys and, if no duplicate is found, stored in a separate index for analysis. In the case of duplicates, the existing entry is extended, which allows us to analyze public key reuse across multiple user accounts and platforms.

Using this methodology, we retrieve two raw data sets by scraping publicly available SSH keys from GitHub, GitLab, and Launchpad, the first in June 2023 and the second in January 2025 (Table 2).

3.2 Security of SSH Client Keys

We analyze each retrieved key according to the rules stated by the CA/Browser forum [28, Sections 6.1.1.3, 6.1.5, 6.1.6]. Although these rules are only intended for TLS server certificates, we believe long-lived SSH client key pairs should also fulfill those requirements related to the security of the key material. We also analyze the set of all RSA keys for common prime factors using the batch GCD attack. To the best of our knowledge, together, these tests cover all known, efficient attacks against public keys that can be run against a large database of keys. Other tests, for example, those found additionally in the RsaCtfTool,[3] are too slow to be run against a large dataset.

Short RSA keys. [28, Section 6.1.5] proposes that the encoded modulus length must be at least 2048 bits and fit byte boundaries. If the modulus $N = pq$ is too small, a brute-force attack that factors N can recover an RSA private key from the public key. State-of-the-art is a number field sieve attack, such as CADO-NFS [61]. The largest RSA modulus currently factored is RSA-250 from the RSA factoring challenge [34] and has 829 bits [19]. It was factored by Boudot et al. in 2020, using 2700 CPU-core-years. We consider all keys up to that size broken by brute-force factoring.

RSA Public Exponents. [28, Section 6.1.6] proposes that the public exponent e must be an odd integer larger or equal to 3 and should be in the range $[2^{16} + 1, 2^{256} - 1]$. A key that is not compliant with this rule is suspect, because a large public exponent e (i.e., roughly the same length as the modulus) may indicate that a small d was chosen during key generation, and e was computed using the extended Euclidean algorithm. Since small values of d can be found by exhaustive search, we check if the set of public RSA keys contains any large public exponent.

RSA Batch GCD and Small Prime Factors. In 2012, Heninger et al. [33] published an attack using a batch GCD algorithm to efficiently find prime factors of individual keys in large sets of RSA keys. The attack exploits cases where RSA keys share a common prime factor. We use an enhanced batch GCD algorithm incorporating subtrees to optimize memory usage, following an approach similar to that proposed by Hastings et al. [32]. Our implementation enables the subsequent addition of new keys and the testing of individual keys, with or without their inclusion in the dataset.

According to [28, Section 6.1.6], the modulus N should not have (prime) factors smaller than 752. We extend this and check for the first one million prime factors (p_{last} = 15,485,867) by adding their product during batch GCD computation. For all RSA moduli with one or multiple factors in the first ten thousand primes (p_{last} = 104,743), we apply the elliptic curve factorization algorithm by Lenstra [39] with conservative parameters suitable for finding factors up to 25 decimal digits, as previously done by Cryptosense [24].

Debian, ROCA, and Fermat Vulnerabilities. [28, Section 6.1.1.3] states that keys should be checked for known weak keys, such as Debian weak keys [11], ROCA weak keys [48], and Fermat weak keys [15]. In 2008, the OpenSSL version in Debian GNU/Linux had an inadequate procedure to initialize the random number generator. As a result, all RSA and (EC)DSA keys generated with such a system lack entropy and can be brute-forced with only a few hours of CPU time. The badkeys project[4] by Hanno Böck includes precomputed tables for 1024-bit DSA keys generated by OpenSSH, ECDSA keys under the NIST P-256 and P-384 curves generated by OpenSSL, as well as RSA keys of size 1024, 2048, 3072, and 4096-bit generated by OpenSSL and OpenSSH. We use code from the badkeys project to check for these keys while simultaneously testing for other known compromised keys. ROCA [48] describes a type of keys whose prime numbers are generated with reduced entropy. ROCA keys can be detected efficiently with a negligible false positive rate. We use the tool from the original authors to test the keys in our database. The Fermat attack [15] can find p and q by brute-force search around

[2] As a practical demonstration, Filippo Valsorda maintains a special SSH server `whoami.filippo.io`, that identifies users trying to connect to it by comparing their public key to a mirror of GitHub's user database.

[3] https://github.com/RsaCtfTool/RsaCtfTool, accessed: 2025-04-14

[4] https://badkeys.info/, accessed: 2025-04-14

$\sqrt{N}$ if p and q are close. To achieve this, the attack tries to represent $N = pq$ as the difference of two squares $N = (a+b)(a-b) = a^2 - b^2$ for some a and b. To do so, a is initialized to the largest integer $\leq \sqrt{N}$. Then, in what is considered a round in the attack, it is checked whether $a^2 - N$ is a perfect square. If not, a is incremented by 1, and the attack continues with the next round. We apply the Fermat attack with 100 rounds.

(EC)DSA. For DSA, we perform an explicit domain parameter validation with partial validation of the generator and a full public key validation as described in [7, Section 4.1] and [7, Section 5.3.1], respectively. Any prime p with fewer than 2048 bits is considered weak. A prime with fewer than 795 bits, the current record of discrete logarithm computation for safe primes [18], is considered broken. Additionally, we check for compromised keys, such as Debian weak keys, using the badkeys tool.

For ECDSA, we first validate that NIST P-256, P-384, or P-521 is used as the underlying elliptic curve following [28, Section 6.1.5] and that the public key is a valid encoding of an elliptic curve public key. Then, we perform an ECC full public key validation routine described in [8, Section 5.6.2.3.3] per [28, Section 6.1.6]. This ensures that the public key point is not the identity element, that the coordinates are within valid bounds, that the public key is a point on the curve, and that it has the correct subgroup order. Additionally, we check for compromised keys, such as Debian weak keys, using the badkeys tool.

EdDSA. For EdDSA public keys, to the best of our knowledge, there are no official security considerations or attacks to consider. Nevertheless, we validate the public key following the decoding process described in [37, Section 5.1.3], and by calculating the public key point's order. As the decoding process occurs on every signature validation, we expect that keys not passing this test will be unusable in practice. Additionally, we check for compromised keys, such as Debian weak keys, using the badkeys tool.

3.3 Evaluating Public Key Upload Restrictions

To test for existing public key upload restrictions put forth by the platforms, we generate test keys that specifically fail our criteria described in Section 3.2. Each test key should violate a single test criterion to allow for fine-grained detection. Note, however, that this cannot be achieved for all requirements; for example, a key with invalid ECDSA encoding cannot pass any subsequent test. Each test key is then uploaded to the platforms, and the response is recorded. A successful upload is indicated by the platform listing the public key under the user's public keys; a successful authentication is not required.

3.4 Evaluating SSH Clients in the Lab

We restrict the evaluation of client capabilities to the supported signature algorithms because they determine the strength of the client's authentication.

Selection of Clients and Agents. We compile a list of SSH clients for evaluation (Table 4) from a comparison of SSH implementations assembled by Max Horn,[5] a Wikipedia page on SSH clients,[6] and implementations well-known to us. We exclude clients that are no longer actively maintained, as well as clients supporting neither ECDSA nor EdDSA. Evaluation is limited to each client's most recent version, and 24 clients in total; evaluating historical versions is out of scope.

For SSH agents, no comparative lists exist. The list in Table 5 is based on internet search results, and includes all agents mentioned on the Wikipedia page on SSH agents.[7]

Determining Client Features. For each SSH client implementation, we gather the supported signature algorithms based on the documentation. We also identify the underlying cryptographic library using basic binary analysis, process monitoring, and publicly available information on the vendor's website. For binary analysis under Linux, we use a combination of the tools `ldd` for identifying required shared libraries, `nm` for listing static and dynamic symbols, and `strings` for searching character sequences. Similarly, for binary analysis under Windows, we utilize the `dumpbin` and `strings` tools provided by Microsoft. Our findings are detailed in the additional material in the artifacts.

Tests for Deterministic/Biased Nonces. This evaluation is limited to the DSA, ECDSA, and EdDSA algorithms due to the use of a nonce, where even a slight bias in nonce generation can allow for efficient recovery of the secret key. While major SSH implementations have disabled DSA support by default, and the OpenSSH project intends to remove DSA support from its source code entirely in early 2025, some clients still support it for compatibility reasons.

We implement a novel tool in Go that determines the nonce generation method and potential nonce bias, even for proprietary clients without available source code. To do so, the tool captures DSA, ECDSA, or EdDSA client authentication signatures by starting a custom SSH server. If supported by the SSH client, the client is automatically invoked to connect to the server. Otherwise, the connection is set up manually or by using UI automation tools, such as an automatic mouse clicker. We have access to the client's private key, as this tool is designed for in-lab analysis. With the help of this private key, the nonce values are recovered from the sampled signatures and analyzed.

To check for known deterministic nonce schemes from RFC 6979 and PuTTY 0.80 (k_{proto}), our tool generates two nonces according to these schemes and compares them with the nonce value from the received signature. To check for random nonces, the tool enforces further authentication attempts within the same connection, using the same client public key, by indicating a partial authentication success. Partial success is part of the SSH standard and allows for in-protocol, multifactor authentication by sending `USERAUTH_FAILURE` with a special boolean flag set. Depending on their internal logic, a client may try to authenticate twice or more using the same public key in the same session. Such a client computes the signature twice over the same payload. If these two signatures are identical, we have detected a new, unknown deterministic nonce scheme. Otherwise, we can conclude that the nonce generation is at least partially random.

[5] https://ssh-comparison.quendi.de/, accessed: 2025-04-14
[6] https://wikipedia.org/wiki/Comparison_of_SSH_clients, accessed: 2025-04-14
[7] https://wikipedia.org/wiki/Ssh-agent, accessed: 2025-04-14

Our tool can also test for biased nonce generation, applying two statistical tests: a single-bit Z-test to detect any single-bit bias, and a more sophisticated Rayleigh test based on an implementation by Daniel Bleichenbacher in Project Wycheproof [8] to detect a possible multi-bit bias. When collecting nonces for this analysis, we do not indicate partial success to avoid introducing bias by repeatedly sampling the same nonce, and instead sample each signature over a fresh payload. The significance level of the test is set to $\alpha = 2^{-32}$ to avoid false positives. For each implementation, a minimum of 10,000 nonces is sampled for bias analysis.

4 SSH Client Keys

4.1 Longitudinal Analysis of SSH Client Keys

We collected over 30M SSH client keys from GitHub, GitLab, and Launchpad in two scans in June 2023 and January 2025 (Table 2). We found that the distribution of algorithms differs considerably among the platforms, with Launchpad users predominantly using RSA keys (92.22% / 90.40%) and DSA keys (5.77% / 5.48%). In contrast, GitHub and GitLab users also use Ed25519 (37.46% / 48.24% and 20.08% / 23.70%). Most notably, the percentage and absolute number of Ed25519 keys on GitHub are rising sharply by over 10% for our datasets, leading to an almost even split between RSA and Ed25519 keys in January 2025. A similar trend can also be observed for GitLab and Launchpad; however, it is less pronounced and far from being split evenly. At the same time, the absolute number of RSA keys on GitHub is receding.

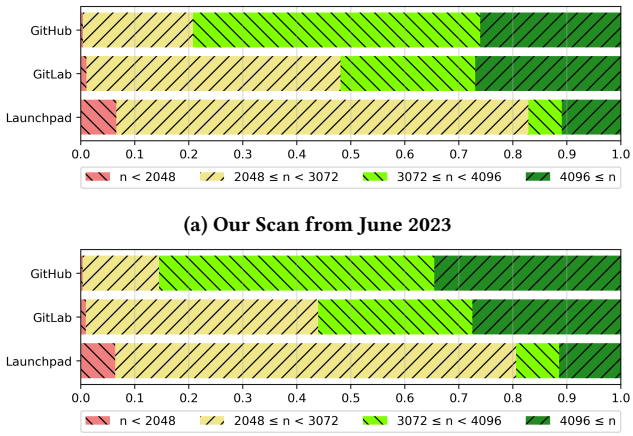
Compared to earlier reports, we see an increased adoption of elliptic curve algorithms, particularly Ed25519. In 2015, Cartwright-Cox [22] published data on 1.38M GitHub user keys, of which 1.35M (97.67 %) are RSA keys, and the remaining 2% DSA, with only 859 ECDSA and 210 Ed25519 keys. A 2018 study by Romailler et al. [4] collected 4.7M GitHub and 1.2M GitLab user keys as part of a much larger dataset of 346M keys. Of that larger dataset, 94.48% are RSA keys, and less than 1% are DSA keys. In contrast, we see a strong adoption of Ed25519 paired with a deprecation of DSA on GitHub and GitLab.

Two-Factor Authentication. Our analysis is based on the cryptographic primitive underlying the SSH key and does not distinguish between regular keys and those stored on U2F devices, as described in [47]. From the SSH algorithm name, we observe sporadic use of Ed25519 and ECDSA NIST P-256 U2F keys on GitHub and GitLab only, accounting for approximately 3.5% of ECDSA NIST P-256 keys and less than 1% of Ed25519 keys.

RSA. Figure 3 shows that on GitHub in January 2025, 99.62% of all RSA keys have a modulus of length at least 2048 bits; this number only amounts to 93.64% on Launchpad, leaving GitLab in between at 99.08% in the same time range. A trend towards more secure RSA modulus bit lengths can be observed on all platforms, although it is more noticeable on GitHub and GitLab.

As for the public exponent, $e = 2^{16}+1$ is the most common choice by a significant margin, accounting for 96.55% of all exponents in both datasets. Other common choices are $e = 37$ (2.12%), used by PuTTY up to version 0.75, and $e = 35$ (1.32%), used by OpenSSH up to version 5.4. Other values are rarely used for more than one key.

[8] https://github.com/C2SP/wycheproof, accessed: 2025-04-14



(a) Our Scan from June 2023

(b) Our Scan from January 2025

Figure 3: The relative distribution of RSA modulus bit lengths n in user keys across each platform shows that GitHub users, by percentage, use the strongest keys. The percentage of longer bit lengths increased measurably between our scans.

The longest RSA public exponent observed has 96 bits alongside a 2158-bit modulus and was found on GitLab.

ECDSA. Among all ECDSA keys in our datasets, the most commonly used curves are NIST P-256 and NIST P-521, as seen in Figure 4. The only other curve in our dataset is NIST P-384, with a minor share, thus reflecting the three common choices for ECDSA curves in SSH. Interestingly, we observe a shift from NIST P-521 towards less secure NIST P-256 keys on all platforms.

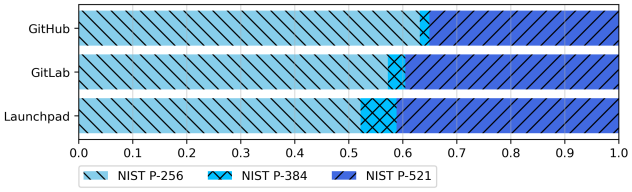
DSA. We observed only a few thousand DSA public keys available for authentication, none of them on GitHub (cf. Section 4.3). Between our scraping runs, the number of keys on GitLab decreased, while the number on Launchpad stagnated. 88.57% of all DSA keys use a 1024-bit modulus with a 160-bit subgroup, in line with the specification. However, 10.02% of keys use moduli of larger bit lengths, most commonly 2048 bits, albeit maintaining the 160-bit subgroup. Using larger moduli sizes is not standard-compliant but is often supported by SSH implementations as long as the subgroup bit length remains unchanged. Due to generic discrete log algorithms (e.g., Pollard's kangaroo [56] algorithm), increasing the modulus size without increasing the subgroup size does not raise the effective security level.

4.2 Security of SSH Client Keys

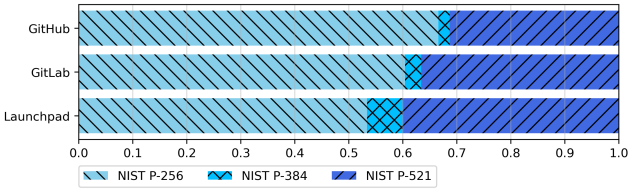
Short RSA keys. In the June 2023 dataset, we found 77,411 keys with modulus bit lengths shorter than 2048 bits. Of those, 93 keys are shorter than 830 bits, putting them at risk of being factored by computing resources available today. A single key on GitLab was only 8 bits long ($N = 13^2$); probably a test key to probe upload filters on GitLab. As for byte boundary bit lengths, we observe 20,240 keys with a bit length that is not a multiple of 8, potentially causing compatibility issues. About half of these keys are of length $n = 1023$ or $n = 2047$, possibly resulting from a bug in PuTTY (fixed

Table 2: Distribution of asymmetric key algorithms among the SSH client keys on popular source repository platforms.

	2015 [22]	Our Scan from June 2023				Our Scan from January 2025			
Algorithm	GitHub	GitHub	GitLab	Launchpad	Unique	GitHub	GitLab	Launchpad	Unique
RSA	97.67%	61.53%	79.44%	92.22%	68.40%	50.82%	75.79%	90.40%	60.80%
	1,205,330	5,660,641	4,595,059	132,215	9,959,924	5,338,842	5,272,676	136,743	10,372,614
Ed25519	0.02%	37.46%	20.08%	1.41%	30.74%	48.24%	23.70%	3.41%	38.39%
	210	3,446,461	1,161,821	2,026	4,476,970	5,066,902	1,649,163	5,163	6,548,889
ECDSA	0.07%	1.01%	0.40%	0.60%	0.77%	0.94%	0.44%	0.71%	0.74%
	859	92,903	23,052	860	112,566	98,350	30,774	1,076	125,593
DSA	2.24%	0.00%	0.08%	5.77%	0.09%	0.00%	0.07%	5.48%	0.07%
	27,683	0	4,757	8,273	12,924	0	4,676	8,282	12,858
Total	1,234,082	9,200,005	5,784,689	143,374	14,562,384	10,504,094	6,957,289	151,264	17,059,954



(a) Our Scan from June 2023



(b) Our Scan from January 2025

Figure 4: The relative distribution of curves used with ECDSA algorithms per platform shows that Launchpad users, by percentage, use more secure ECDSA keys. We observe a shift from NIST P-521 towards NIST P-256 between our two scans.

in version 0.72) that caused some generated RSA keys to be one bit shorter than intended.

We observed similar numbers in January 2025, indicating that the number of short RSA keys has not increased significantly and that users nowadays primarily upload long keys. We found 77,438 keys shorter than 2048 bits, of which 98 are shorter than 830 bits, and 20,692 keys where the modulus bit length is not a multiple of 8. When comparing the two datasets, we found 60,493 shared keys to be shorter than 2048 bits and 90 of those to be shorter than 830 bits. Similarly, 15,777 keys with a bit length not divisible by 8 appear in both datasets.

RSA Public Exponents. In January 2025, we found four keys with even exponents, three of which have $e = 2^{16} + 2$, and the fourth has a 96-bit exponent, resulting from a corrupt RSA public key; no even exponents were observed in our June 2023 dataset, and thus no such key is shared between the two datasets. In both datasets, all odd exponents had at most 64 bits.[9] We conclude that there is no significant probability for keys with small d.

RSA Batch GCD and Small Prime Factors. In the June 2023 dataset, we identified 305 keys with a GCD greater than one. Among these, 56 were correctly generated keys that reused one of their primes, compromising their security. The remaining 249 keys exhibited small factors, with the largest GCD being 1,842,374,275. Many of the GCDs were smaller, and a substantial portion were composite. Looking only at the first one million prime numbers, we found 306 keys that contained at least one such small prime as a factor. Counting all keys with a GCD greater than one or having a small prime factor, we get a total number of 362 (0.004%) such keys.

In the January 2025 set, we found 128 keys with a GCD greater than one. Unlike the first set, none of these were correctly generated RSA keys with only two large prime factors. Instead, all keys in this set exhibited small factors, with the largest GCD being 1,300,062,075. Similar to the first set, most GCDs were smaller and predominantly non-prime. Furthermore, 149 keys contained one or more of the first one million prime factors in their modulus. These keys include all 128 keys found with batch GCD, so the total number of keys with a GCD greater than one or having a small prime factor remains 149 (0.001%). Of those, 19 keys were found in both datasets.

A potential reason for keys with one or multiple small prime factors can be an accidental change to the Base64 encoding of the public key during upload (e.g., a single Base64 character being altered), rendering those keys unusable *for the legitimate user*. However, we suppose that these keys can still be used for authentication *by an attacker* if the attacker computes a signature using a secret exponent d calculated with the correct value for $\varphi(N)$. From these keys, we selected all keys with a prime factor in the first ten thousand primes, yielding 396 unique keys, 281 keys from June 2023, and 132 keys from January 2025, with 17 keys shared between both datasets. We attempted ECM factorization up to 25-decimal-digit factors on those 396 keys. These attempts returned complete factorizations for 29 keys. Where only partial factorization was successful, the modulus was reduced by 99.53 bits on average.

[9] This is also the limit of public exponent bits for large key sizes in OpenSSL; see https://github.com/openssl/openssl/blob/master/include/openssl/rsa.h#L56, accessed: 2025-04-14

Debian, ROCA, and Fermat Vulnerabilities. Our June 2023 dataset contained 93 keys affected by the ROCA vulnerability, compromising their security. All affected keys were found on GitLab or Launchpad. In our January 2025 dataset, this number dropped significantly for GitLab while increasing by one for Launchpad to 25 vulnerable keys across both platforms, 17 of which were already included in our June 2023 dataset. The significant decrease in vulnerable keys on GitLab is likely a response to our first disclosure.

Fermat factorization yielded a single result in our June 2023 dataset, where $N = 13 \cdot 13$. Apart from this, the Fermat test with 100 rounds did not return factorizations for any of the other RSA moduli, confirming the negative findings by Böck [15].

Additionally, we checked every RSA public key against each *blocklist* defined by the badkeys tool. These blocklists include Debian weak keys for several applications and key sizes, keys vulnerable to CVE-2021-41117, keys leaked in Git repositories and websites, and test keys from RFCs and software tests. In our June 2023 dataset, we found 176 keys on at least one blocklist. Of these, 73.30% were blocked as Debian weak keys. Of all keys on Launchpad, 0.08% are Debian weak keys. This percentage is several orders of magnitude larger than that on the other platforms. The number of keys in a blocklist slightly decreased to 133 RSA keys in our January 2025 dataset, 84.96% being Debian weak keys. 123 keys are contained in both datasets, of which 106 keys are found on Launchpad, indicating a reduction in blocklisted keys on GitHub and GitLab.

(EC)DSA. For DSA, we found the same six keys with a bit length of p less than 795 bits on Launchpad in each of the two datasets, which puts them at risk of a discrete logarithm attack with the computing resources available today. Our January 2025 dataset contains one 1024-bit DSA key from Launchpad where p is not prime and g and y do not have order q, indicating a corrupt public key. In the same dataset, three unique keys have a subgroup size of 256 bits rather than 160 bits, two of which were already present in our June 2023 dataset. The different subgroup size renders the keys unusable in practice due to the lack of length fields in the DSA signature blob in SSH [44, Section 6.6]. As for Debian weak keys, we found a total of 35 compromised DSA keys across both datasets; none of these keys are shared by different users. However, 29 of these keys appear in both datasets.

For ECDSA, all keys from both datasets passed our validity checks, and the badkeys tool does not find any key on any badkeys blocklist, including Debian weak keys.

EdDSA. For EdDSA, we found 327 keys in our June 2023 dataset with an invalid encoding and 315 with a higher-than-expected point order. These numbers increased to 759 invalid encodings and 641 higher point orders in January 2025, where 82 invalid encodings and 80 higher point orders from the June 2023 dataset were also included in the January 2025 dataset. The significant increase in invalid Ed25519 keys is likely caused by the increase in overall Ed25519 keys. Similar to RSA keys, both results may be related to accidental modifications of the Base64-encoded public key during upload. No key with a lower-than-expected point order was observed. Six distinct keys are listed on at least one badkeys blocklist; only one of which is present in both datasets.

Table 3: Public key upload restrictions on the Git-based repository hosting services GitHub, GitLab, and Launchpad in January 2025. A cross (✗) indicates that the platform prevents the key upload, while a check mark (✓) indicates that the upload is possible. For DSA, p is the prime, q is the order of the subgroup, g is the generator, and y is the public key. For RSA, $N = pq$ is the modulus of bit length n, and e is the exponent. For ECDSA, the public key is (x, y).

	Test Key					Test Key			
RSA	Valid	✓	✓	✓	DSA	Valid	✗	✗	✓
	$n = 1024$	✓	✓	✓		2048-bit p	✗	✗	✓
	$n = 1023$	✓	✓	✓		256-bit q	✗	✗	✓
	$n = 1022$	✗	✓	✓		p not prime	✗	✗	✓
	$n = 512$	✗	✓	✓		q not prime	✗	✗	✓
	$8 \nmid n$	✓	✓	✓		$g \geq p$	✗	✗	✗
	Even Modulus	✗	✓	✓		$y \geq p$	✗	✗	✓
	Even Exponent	✓	✓	✗		$ord(g) \neq q$	✗	✗	✓
	$e < 2^{16} + 1$	✓	✓	✓		$ord(y) \neq q$	✗	✗	✓
	$N = p^2$	✓	✓	✓		Debian Weak Key	✗	✗	✓
	Small Factor	✗	✓	✓					
	Debian Weak Key	✓	✓	✓					
	CVE-2021-41117	✗	✗	✓	ECDSA	Valid	✓	✓	✓
	ROCA	✗	✓	✓		Invalid Encoding	✗	✗	✗
	Fermat	✓	✓	✓		Point at Infinity	✓	✗	✗
						$x \geq p$	✗	✗	✗
Ed25519	Valid	✓	✓	✓		$y \geq p$	✗	✗	✗
	Invalid Encoding	✓	✓	✓		Not on Curve	✗	✗	✗
	Point Order $\neq q$	✓	✓	✓		Debian Weak Key	✓	✓	✓

4.3 Public Key Upload Restrictions

Our generated test key dataset contains 41 test vectors, including one valid public key per algorithm family. Our results indicate that GitHub does not allow uploading DSA keys, and while some user accounts on GitLab still have DSA keys configured, uploading new DSA keys to GitLab is also no longer possible. For RSA, GitHub prevents uploading short RSA keys with fewer than 1023 bits, keys vulnerable to ROCA, and keys with small factors. Most test vectors for ECDSA were rejected by all platforms, including those with invalid encoding, invalid coordinate bounds, and an invalid point returned invalid key errors. The point at infinity was accepted by GitHub as a public key but rejected by the other two platforms. Only Launchpad returned additional invalid key errors on an out-of-bounds generator for DSA and an even RSA exponent. No upload restrictions were detected for Ed25519 keys on all platforms. The detailed results for all keys can be found in Table 3.

5 SSH Client Signatures

5.1 SSH Client Features

Table 4 summarizes the basic features of the 24 SSH clients we investigated. The most commonly used crypto libraries were OpenSSL (8), PuTTY (4), and LibreSSL (2)—a fork of OpenSSL. All other libraries appeared only once. All clients support using SSH agents for client authentication; nine are bundled with an agent implementation. All but three libraries support agent forwarding, and two have activated it by default. With regard to nonce generation, we verified

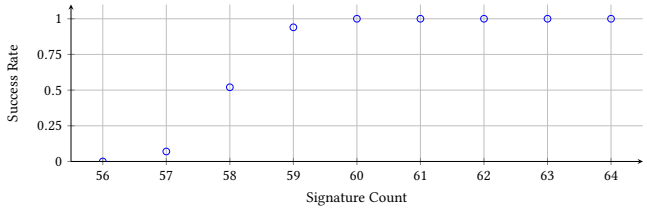


Figure 5: Success rate of the biased nonce attack on PuTTY ECDSA P-521 using USVPPredSieve [2], for a given number of signatures (1024 trials each).

that three clients use deterministic nonces for DSA and ECDSA. For three clients, we could not determine whether deterministic nonces are used, as they do not attempt to authenticate multiple times with the same key in the same connection. However, we can rule out either one using k_{proto} or RFC 6979 for nonce generation. Regarding biased nonces, we find that the upper nine bits of nonces in PuTTY 0.80 ECDSA NIST P-521 signatures are zero, indicating a 9-bit bias. Apart from PuTTY, we did not find any nonce bias.

For SSH agents (Table 5), the most commonly used libraries are OpenSSL (4), PuTTY (3), and Libgcrypt (2). Three use deterministic nonces, and 1Password does not support (EC)DSA. Some Git clients are bundled with SSH clients. This might indicate a slight bias in our dataset towards keys generated by bundled clients.

5.2 PuTTY P-521 Private Key Recovery

PuTTY's Deterministic Nonce Generation (k_{proto}). PuTTY generates nonces deterministically to protect against bad random number generation that could result in biased nonces. The initial implementation in 2001 was designed for DSA, but not ECDSA, and generated the nonce by reducing a 512-bit pseudorandom number k_{proto} to a number in the interval $[2, q-1]$. k_{proto} is derived from a label, the private key x, and the message digest $\mathrm{H}(m)$.[10]

$$\begin{aligned} \text{digest} &:= \text{SHA-512}\,(\mathit{label}\,||\,x) \\ k_{\text{proto}} &:= \text{SHA-512}\,(\text{digest}\,||\,\mathrm{H}(m)) \\ k &:= \left(k_{\text{proto}} \bmod q-2\right)+2 \end{aligned}$$

For DSA, q is the order of the cyclic subgroup of $\mathbb{Z}_p^*$ generated by the basis element g and is 160 bits in the case of SSH. When support for ECDSA was added in 2014, the code for k_{proto} was reused, except for using a different label. In ECDSA, q is the order of the elliptic curve used for all computations generated by the base point P.

Bias in P-521. For DSA, and ECDSA with NIST P-256 and P-384, the bias on the nonces k generated with k_{proto} is negligible. However, for P-521, the output of SHA-512 is always smaller than the size of an element of the underlying field $GF(2^{521}-1)$, and thus the value k_{proto} is not reduced. As a result, the nonce $k < 2^{512}$ has the upper nine most significant bits set to zero.

[10] https://github.com/github/putty/blob/7003b43963aef6cdf2841c2a882a684025f1d806/sshdss.c#L337, accessed: 2025-04-14

Hidden Number Problem. In ECDSA, this nonce k is used in the generation of the signature (r, s) on a message m using the private key a as follows: $r = \mathrm{pr}_x(kP)$ and $s = k^{-1} \cdot (h(m) + a \cdot r) \bmod q$, where pr_x is the x-coordinate of a point. Rewriting this yields

$$s^{-1} \cdot r \cdot a + s^{-1} \cdot h(m) \equiv k \pmod q.$$

Setting $t = s^{-1} \cdot r$, $b = s^{-1} \cdot h(m)$, and $\alpha = a$ results in the equivalence relation of the *Hidden Number Problem* (HNP), as formulated by Boneh and Venkatesan [17],

$$t \cdot \alpha + b \equiv k \pmod q.$$

Here, b and t are values known to the attacker, the hidden number α is the secret key the attacker wants to recover, and k is partially known to the attacker, as its nine most significant bits are 0. Given "enough" equivalences, the HNP can be solved, i.e., the value $\alpha = a$ can be computed. The number of HNP equivalences needed to solve the HNP problem (i.e., to compute the private ECDSA signing key) depends on the algorithm used.

Lattice Algorithms to Solve the HNP. For our experiments, we used an algorithm based on lattices from [2, USVPPredSieve] (see also [20]), which can be directly applied to our setting and recovers the private key by solving the Shortest Vector Problem in the constructed lattice. The results are shown in Figure 5. Given 58 ECDSA signatures generated with k_{proto} and P-521, we can compute the signer's private key with a probability greater than 0.5. Given 60 such signatures, we can *always* compute the private signing key. We repeated the experiment 1024 times for each signature count, and the computation time for each trial was at most 67 seconds.

Root Cause Analysis. For the curves P-192, P-224, P-256, and P-384, the bit lengths of the coordinates of the curve points were carefully aligned both with byte boundaries and with the output lengths of hash functions: Both hash function families SHA-2 and SHA-3 have hash value lengths of 224, 256, and 384 bits, and the output length 192 appears in NIST SP 800-208 as a truncated version of SHA-256. The only exception to this rule is P-521, for using a Mersenne prime. Informed programmers wonder about possible typos (512 vs. 521), and discuss why this parameter was chosen.[11]

6 Related Work

SSH Client Authentication. Password-based authentication of SSH clients has been studied previously by Owens and Matthews, and Song et al. [51, 60]. Javed and Paxson [36] statistically measured and evaluated distributed online dictionary attacks against password-based authentication in SSH. To identify client authentication efforts using a stolen SSH client password, Piet et al. [54] used keystroke authentication. Park et al. [52] analyzed router log data to detect SSH brute force attacks. SSH honeypots were used by Melese and Avadhani, Koniaris et al., and Wu et al. [38, 45, 63] to study brute-force online dictionary attacks. In this paper, we study the security of SSH client authentication with digital signatures.

[11] https://crypto.stackexchange.com/questions/6219/why-do-the-elliptic-curves-recommended-by-nist-use-521-bits-rather-than-512, accessed: 2025-04-14

Table 4: Overview of SSH clients with their respective versions and operating systems for evaluation. For each client, the table lists whether source code is publicly available (), the client ships with an SSH agent implementation () or supports the use of agents for authentication (), and whether agent forwarding is supported (). A bolt () indicates that agent forwarding is enabled by default, which we consider dangerous. Additionally, for each implementation, the table includes the determined cryptographic library and nonce scheme. For some clients, capturing two signatures for the same plaintext failed due to disconnects after the first authentication attempt. In such cases, we can only rule out known deterministic schemes.

				SSH Agent				
Name	Version	OS					Cryptographic Library	Nonce Scheme
AbsoluteTelnet	12.16		○	○	●	ϟ	Crypto++	*Not* RFC 6979 / k_{proto}
AsyncSSH	2.18.0		●	○	●	●	OpenSSL 3.3.2	Random
Bitvise	9.42		○	○	●	●	Windows CNG	*Not* RFC 6979 / k_{proto}
Cyberduck	9.0.1		●	○	●	○	BC Java 1.78.1	Random
Dropbear	2024.86		●	○	●	●	LibTomCrypt 1.18.2	Random
Erlang/OTP SSH	5.2.1		●	○	●	○	OpenSSL 3.3.2	*Not* RFC 6979 / k_{proto}
FileZilla	3.67.0		●	○	●	○	PuTTY 0.73	Random
Golang x/crypto/ssh	0.29.0		●	○	●	●	Go crypto 1.23.3	Optional (Any)
libssh	0.11.1		●	○	●	●	OpenSSL 3.3.2	Random
							MBedTLS 3.6.2	RFC 6979
OpenSSH Portable	9.9p1		●	●	●	●	OpenSSL 3.3.2	Random
			●	●	●	●	LibreSSL 3.3.6	Random
Paramiko	3.5.0		●	○	●	●	OpenSSL 3.3.2	Random
PKIX-SSH	15.3		●	●	●	●	OpenSSL 3.3.2	Random
PortX	2.2.12		○	○	●	●	OpenSSL 1.1.1	Random
PuTTY	0.80		●	●	●	●	PuTTY 0.80	k_{proto}
	0.81		●	●	●	●	PuTTY 0.81	RFC 6979
SecureCRT	9.5.2		○	●	●	●	BSAFE Crypto-C ME 4.1.5	Random
Secure ShellFish	2025.17		○	●	●	●	OpenSSL	*Not* RFC 6979 / k_{proto}
ServerCat	1.18		○	○	○	○	BoringSSL	*Not* RFC 6979 / k_{proto}
SSH Term	7.0.26		○	○	○	○	OpenSSL 3.5.0	*Not* RFC 6979 / k_{proto}
Tectia SSH	6.6.3.490		○	●	●	ϟ	*Proprietary*	Random
							OpenSSL 3.0.8 (FIPS)	Random
Tera Term	5.2		●	○	●	●	LibreSSL 3.8.2	Random
Termius	9.8.5		○	●	●	●	Libsodium 1.0.17	Random
	9.21.2		○	●	●	●	Libsodium 1.0.17	Random
Win32 OpenSSH	9.5.0.0		●	●	●	●	LibreSSL 3.8.2	Random
WinSCP	6.3.4		●	○	●	●	PuTTY 0.81	RFC 6979
XShell 7	0170		○	●	●	●	OpenSSL 1.0.2u	Random

SSH Client Keys. In 2015, Cartwright-Cox [22] scraped 1.2M keys of GitHub users, found 49 RSA keys shorter than 1000 bits, and reported some users whose keys were generated on a Debian system with weak randomness. In 2023, Böck [15] tested the Fermat attack against this data set and found no vulnerable keys. In 2016, Barbulescu et al. [6] analyzed weak RSA keys on the internet, including 3M RSA keys from GitHub users. They found one 512-bit key, but could not find any common factors using batch GCD among the other keys. Also, in 2015, Cryptosense [24] found small factors in GitHub user keys and could factor 12 keys with 2048 bits using GMP-ECM by Zimmermann.[12] In 2018, Amiet and Romailler [4] analyzed 4.7M keys from GitHub and 1.7M keys from GitLab for ROCA vulnerabilities and common factors using batch GCD. They found vulnerable keys in their dataset, but did not separate results by data source. In this work, we collect keys from GitHub, GitLab, and Launchpad. After reviewing the related work and relevant standards, we identified a broad range of security issues. Based on these issues, we implemented—to the best of our knowledge—the most comprehensive suite of security tests for the detection of potential security issues and vulnerabilities.

SSH Signature-based Authentication. SSH uses the PKCS#1 v1.5 signature scheme for authentication with RSA keys. Yahyazadeh et al. [64] formalized the PKCS#1 v1.5 signature specification and developed a black-box differential testing framework targeting this signature scheme. After analyzing 45 libraries, they identified various conformance issues and six libraries susceptible to Bleichenbacher's signature forgery attacks [26]. These attacks allow an attacker to forge RSA signatures if the verifier does not strictly check the PKCS#1 v1.5 format of the verified signature, and the

[12] https://gitlab.inria.fr/zimmerma/ecm, accessed: 2025-04-14

Table 5: Overview of SSH agent implementations, their underlying cryptographic library, and nonce generation method. 1Password does not support any (EC)DSA algorithms.

Agent	Version	Crypto Library	Nonce
1Password	8.10.56	Libgcrypt 1.10.3	*n/a*
GnuPG	2.4.4	Libgcrypt 1.10.3	RFC 6979
Goldwarden	0.3.6	Go 1.22.1 crypto	Random
KeeAgent	0.13.8	BC C# 1.9.0	Random
MobaXTerm	24.4	PuTTY 0.81	RFC 6979
OpenSSH	9.6p1	OpenSSL 3.0.13	Random
PKIX-SSH	15.3	OpenSSL 3.0.13	Random
PuTTY Pageant	0.80	PuTTY 0.80	k_{proto}
	0.81	PuTTY 0.81	RFC 6979
SecureCRT	9.5.2	BSAFE Crypto-C ME 4.1.5	Random
Tectia SSH	6.6.3.490	*Proprietary*	Random
		OpenSSL 3.0.8	Random
Termius	9.9.0	Libsodium 1.0.17	Random
Win32 OpenSSH	9.5.0.0	LibreSSL 3.8.2	Random
XShell 7 Xagent	0170	OpenSSL 1.0.2u	Random

RSA public key exponent is small (e.g., $e = 3$), which we did not observe in the collected data sets. In our work, we concentrated on the security analysis related to signature generation. Signature verification evaluations, such as those presented by Yahyazadeh et al. [64], could extend in our framework.

Ryan et al. recovered private RSA keys of SSH servers from faulty signatures in [59]. Breitner and Heninger studied ECDSA signatures in cryptocurrencies and SSH, and were able to extract private keys due to biased nonces [20]. By collecting public SSH server keys from the internet, Heninger et al. could compute the private keys of 0.03% of SSH servers using RSA and 1.03% of SSH servers using DSA [33]. In this paper, we use techniques similar to Heninger et al. to analyze the security of SSH *client* keys.

Deterministic Signature Schemes. Cao et al. and Poddebniak et al. [21, 55] showed that deterministic variants of ECDSA and EdDSA are vulnerable to *fault attacks*, where invalid signatures are needed. Please note that our attack on PuTTY is not a fault attack but uses *valid* signatures. We used the algorithm described by Albrecht and Heninger [2] to compute the private key.

Attacks on the SSH Protocol. In 2009, Albrecht, Paterson, and Watson [3] attacked the binary packet protocol of SSH by exploiting the encrypted length field. By measuring the number of ciphertext bytes accepted by the server from the network, they derived a decryption oracle for some bits of an encrypted block. Paterson and Watson [53] showed a variation of this attack. Wei Dai [25] described a theoretical attack against SSH employing CBC-mode ciphers, while Bhargavan and Leurent [13] presented a handshake transcript collision attack using SHA-1. Bäumer et al. [9] showed a prefix truncation attack on the ciphertext for some cipher modes using sequence number manipulation, despite formal security proofs for SSH [1, 10, 53]. Fiterau-Brostean et al. [27] learned Mealy state machines of three SSH server implementations; no weaknesses were discovered.

7 Discussion

Mitigations. Users of PuTTY are advised to update to version 0.81 or newer to avoid biased nonce generation when using ECDSA signatures with NIST P-521 keys. Additionally, users should revoke any NIST P-521 keys used with a PuTTY version below 0.81 by removing them from servers and platforms and regenerating them as necessary.

Implementers of cryptographic libraries need to carefully avoid biased nonces in (EC)DSA. As our attack on the deterministic nonce generation of PuTTY has shown, even deterministic nonce schemes may introduce bias. We recommend the algorithm described in [57] for the generation of deterministic nonces, which can generate nonces of arbitrary length.

Operators of online collaborative development platforms, and more generally server implementers, are advised to improve their public key security baselines in several aspects. First, they should test public keys on upload and reject keys based on the requirements outlined in [28]. To complement this, platforms should perform periodic rescans of all public keys as necessary. While additional upload checks will necessarily add overhead, we estimate that most checks are computationally cheap and therefore reasonable to implement. Platform operators may decide to perform more expensive checks, such as batch GCD, offline.

Furthermore, platforms should reevaluate whether beneficial use cases[13] for publishing their users' public keys outweigh the negative implications posed by privacy concerns, for example account linking, and scraping for weak keys.

On the SSH client side, vendors need to be aware of the design of the SSH agent interface. Obviously, it can be used by malicious servers to forge SSH signatures, potentially bypassing authentication guarantees. However, because it provides unrestricted access to a perfect signing oracle, it can also be vulnerable to attacks aiming for signature forgeries or private key extraction. Such a signing oracle may allow attackers to sign Git commits impersonating the victim, potentially bypassing repository restrictions. This may lead to unauthorized code or files being added to the repository and could facilitate supply chain attacks. When using modern continuous integration pipelines, this can also enable remote code execution on these systems when commits are built automatically. This can be somewhat mitigated by the agent checking the "magic bytes" that are present in Git commits and SSH client signatures. However, since this is a blocklist approach, it may only reduce the attack surface. Also, if side channels exist, for example, through timing, an attacker may use such an oracle to recover the private key used by the SSH agent. To mitigate both potential threats, `SSH_AGENT_CONSTRAIN_CONFIRM` can be set by default for all keys loaded into an agent, requiring user interaction upon signature generation.

Impact of Weak Keys. We discovered a variety of weak keys, including RSA keys with fewer than 830 bits and others that were easily factorable—some of which may appear to be merely dummy keys. However, regardless of the specific weakness, a weak key allows an attacker to authenticate itself against the SSH service of

[13] For example, the Ubuntu Linux installer can fetch the user's SSH public key from GitHub using only their GitHub username, providing a convenient way to install their public key on a new system, where copy&paste may not be available.

a Git-based platform as the affected user. This results in a direct relationship between the impact a weak key can potentially have and the repositories and corresponding privileges a user has access to. If, for example, a user uses the platform solely for private development projects, the impact may be straightforward. If, on the other hand, the user has write access to larger software projects or libraries, this can enable complex supply chain attacks by pushing inconspicuous commits to the repository. We did not explore the potential impact of the weak keys we found in greater detail, because finding all repositories a user has access to is non-trivial. For example, the user might have private repositories not visible to the public.

8 Conclusions

In this paper, we shed light on the yet unknown, complex SSH client authentication ecosystem. Among other findings, we show the increasing importance of the EdDSA signature algorithm. We also show the risks associated with proprietary deterministic nonce algorithms in our attack on biased nonces in the ECDSA NIST P-521 implementation of PuTTY.

We have also shown that weak or corrupt SSH public keys can pose a significant threat to the integrity of a user's account. In a longitudinal scan of three source repository platforms, we have found keys compromised by various attacks. We have also shown that upload filters scanning for vulnerable keys can significantly reduce the user's exposure to such vulnerabilities. Platforms that deploy such filters have a substantially lower number of affected weak keys among their users.

Acknowledgments

We thank Kira Höltgen, whose master's thesis included an early version of the key scraper as well as the collection and responsible disclosure of the June 2023 dataset. Fabian Bäumer was supported by the German Federal Ministry of Research, Technology and Space (BMFTR) project "Combinatorial testing of TLS libraries at all levels (KoTeBi)" (16KIS1557). Marcus Brinkmann was supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972.

References

[1] Martin R. Albrecht, Jean Paul Degabriele, Torben Brandt Hansen, and Kenneth G. Paterson. 2016. A Surfeit of SSH Cipher Suites. In *ACM CCS 2016*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM Press, Vienna, Austria, 1480–1491. https://doi.org/10.1145/2976749.2978364

[2] Martin R. Albrecht and Nadia Heninger. 2021. On Bounded Distance Decoding with Predicate: Breaking the "Lattice Barrier" for the Hidden Number Problem. In *EUROCRYPT 2021, Part I (LNCS, Vol. 12696)*, Anne Canteaut and François-Xavier Standaert (Eds.). Springer, Cham, Switzerland, Zagreb, Croatia, 528–558. https://doi.org/10.1007/978-3-030-77870-5_19

[3] Martin R. Albrecht, Kenneth G. Paterson, and Gaven J. Watson. 2009. Plaintext Recovery Attacks against SSH. In *2009 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, Oakland, CA, USA, 16–26. https://doi.org/10.1109/SP.2009.5

[4] Nils Amiet and Yolan Romailler. 2018. Reaping and breaking keys at scale: when crypto meets big data. Presented at DEF CON 2018. https://research.kudelskisecurity.com/wp-content/uploads/2018/08/amiet-romailler-reaping-keys-final-slides.pdf accessed: 2025-09-09.

[5] Christian Aumüller, Peter Bier, Wieland Fischer, Peter Hofreiter, and Jean-Pierre Seifert. 2003. Fault Attacks on RSA with CRT: Concrete Results and Practical Countermeasures. In *CHES 2002 (LNCS, Vol. 2523)*, Burton S. Kaliski, Jr., Çetin Kaya Koç, and Christof Paar (Eds.). Springer Berlin Heidelberg, Germany, Redwood Shores, CA, USA, 260–275. https://doi.org/10.1007/3-540-36400-5_20

[6] Mihai Barbulescu, Adrian Stratulat, Vlad Traista-Popescu, and Emil Simion. 2016. RSA Weak Public Keys Available on the Internet. In *Innovative Security Solutions for Information Technology and Communications*, Ion Bica and Reza Reyhanitabar (Eds.). Springer International Publishing, Cham, 92–102.

[7] Elaine Barker. 2006. *Recommendation for Obtaining Assurances for Digital Signature Applications*. Technical Report NIST Special Publication (SP) 800-89. National Institute of Standards and Technology, Gaithersburg, MD. https://doi.org/10.6028/NIST.SP.800-89

[8] Elaine Barker. 2018. *Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography*. Technical Report NIST Special Publication (SP) 800-56a. National Institute of Standards and Technology, Gaithersburg, MD. https://doi.org/10.6028/NIST.SP.800-56Ar3

[9] Fabian Bäumer, Marcus Brinkmann, and Jörg Schwenk. 2024. Terrapin Attack: Breaking SSH Channel Integrity By Sequence Number Manipulation. In *USENIX Security 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association, Philadelphia, PA, USA. https://www.usenix.org/conference/usenixsecurity24/presentation/b%C3%A4umer

[10] Mihir Bellare, Tadayoshi Kohno, and Chanathip Namprempre. 2002. Authenticated Encryption in SSH: Provably Fixing The SSH Binary Packet Protocol. In *ACM CCS 2002*, Vijayalakshmi Atluri (Ed.). ACM Press, Washington, DC, USA, 1–11. https://doi.org/10.1145/586110.586112

[11] Luciano Bello. 2008. CVE-2008-0166. Available from MITRE, CVE-ID CVE-2008-0166. https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2008-0166 accessed: 2025-07-23.

[12] Daniel J Bernstein. 2004. How to find smooth parts of integers. https://cr.yp.to/papers.html#smoothparts accessed: 2025-04-14.

[13] Karthikeyan Bhargavan and Gaëtan Leurent. 2016. Transcript Collision Attacks: Breaking Authentication in TLS, IKE and SSH. In *NDSS 2016*. The Internet Society, San Diego, CA, USA. https://doi.org/10.14722/ndss.2016.23418

[14] Denis Bider. 2018. Extension Negotiation in the Secure Shell (SSH) Protocol. RFC 8308. https://doi.org/10.17487/RFC8308

[15] Hanno Böck. 2023. Fermat Factorization in the Wild. Cryptology ePrint Archive, Report 2023/026. https://eprint.iacr.org/2023/026

[16] Dan Boneh, Richard A. DeMillo, and Richard J. Lipton. 2001. On the Importance of Eliminating Errors in Cryptographic Computations. *Journal of Cryptology* 14, 2 (March 2001), 101–119. https://doi.org/10.1007/s001450010016

[17] Dan Boneh and Ramarathnam Venkatesan. 1996. Hardness of Computing the Most Significant Bits of Secret Keys in Diffie-Hellman and Related Schemes. In *CRYPTO'96 (LNCS, Vol. 1109)*, Neal Koblitz (Ed.). Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA, 129–142. https://doi.org/10.1007/3-540-68697-5_11

[18] Fabrice Boudot, Pierrick Gaudry, Aurore Guillevic, Nadia Heninger, Emmanuel Thomé, and Paul Zimmermann. 2019. 795-bit factoring and discrete logarithms. https://caramba.loria.fr/dlp240-rsa240.txt accessed: 2025-04-14.

[19] Fabrice Boudot, Pierrick Gaudry, Aurore Guillevic, Nadia Heninger, Emmanuel Thomé, and Paul Zimmermann. 2020. Factorization of RSA-250. https://sympa.inria.fr/sympa/arc/cado-nfs/2020-02/msg00001.html accessed: 2025-04-14.

[20] Joachim Breitner and Nadia Heninger. 2019. Biased Nonce Sense: Lattice Attacks Against Weak ECDSA Signatures in Cryptocurrencies. In *FC 2019 (LNCS, Vol. 11598)*, Ian Goldberg and Tyler Moore (Eds.). Springer, Cham, Switzerland, Frigate Bay, St. Kitts and Nevis, 3–20. https://doi.org/10.1007/978-3-030-32101-7_1

[21] Weiqiong Cao, Hongsong Shi, Hua Chen, Jiazhe Chen, Limin Fan, and Wenling Wu. 2022. Lattice-Based Fault Attacks on Deterministic Signature Schemes of ECDSA and EdDSA. In *CT-RSA 2022 (LNCS, Vol. 13161)*, Steven D. Galbraith (Ed.). Springer, Cham, Switzerland, Virtual Event, 169–195. https://doi.org/10.1007/978-3-030-95312-6_8

[22] Ben Cartwright-Cox. 2015. Auditing GitHub users' SSH key quality. https://blog.benjojo.co.uk/post/auditing-github-users-keys accessed: 2025-04-14.

[23] Lily Chen, Dustin Moody, Karen Randall, Andrew Regenscheid, and Angela Robinson. 2023. Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters. https://doi.org/10.6028/NIST.SP.800-186

[24] Cryptosense. 2015. Batch-GCDing GitHub SSH Keys. https://web.archive.org/web/20161119145246/https://cryptosense.com/batch-gcding-github-ssh-keys/ accessed: 2025-04-14.

[25] Wei Dai. 2002. email to IETF mailing list. https://www.ietf.org/ietf-ftp/ietf-mail-archive/secsh/2002-02.mail accessed: 2025-09-09.

[26] Hal Finney. 2006. Bleichenbacher's RSA signature forgery based on implementation error. https://mailarchive.ietf.org/arch/msg/openpgp/5rnE9ZRN1AokBVj3VqblGlP63QE accessed: 2025-07-23.

[27] Paul Fiterău-Broştean, Toon Lenaerts, Erik Poll, Joeri de Ruiter, Frits Vaandrager, and Patrick Verleg. 2017. Model learning and model checking of SSH implementations. In *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software* (Santa Barbara, CA, USA) *(SPIN 2017)*. Association for Computing Machinery, New York, NY, USA, 142–151. https://doi.org/10.1145/3092282.3092289

[28] CA/Browser Forum. 2024. Baseline Requirements for the Issuance and Management of Publicly-Trusted TLS Server Certificates. https://cabforum.org/working-groups/server/baseline-requirements/documents/CA-Browser-Forum-TLS-BR-2.1.2.pdf accessed: 2025-04-14.

[29] Oliver Gasser, Ralph Holz, and Georg Carle. 2014. A deeper understanding of SSH: Results from Internet-wide scans. In *2014 IEEE Network Operations and Management Symposium (NOMS)*. 1–9. https://doi.org/10.1109/NOMS.2014.6838249

[30] GitHub, Inc. 2025. Using SSH agent forwarding. https://docs.github.com/en/authentication/connecting-to-github-with-ssh/using-ssh-agent-forwarding accessed: 2025-04-14.

[31] Artem Golubin. 2019. Public SSH keys can leak your private infrastructure. https://rushter.com/blog/public-ssh-keys/ accessed: 2025-09-09.

[32] Marcella Hastings, Joshua Fried, and Nadia Heninger. 2016. Weak Keys Remain Widespread in Network Devices. In *Proceedings of the 2016 Internet Measurement Conference* (Santa Monica, California, USA) *(IMC '16)*. Association for Computing Machinery, New York, NY, USA, 49–63. https://doi.org/10.1145/2987443.2987486

[33] Nadia Heninger, Zakir Durumeric, Eric Wustrow, and J. Alex Halderman. 2012. Mining Your Ps and Qs: Detection of Widespread Weak Keys in Network Devices. In *USENIX Security 2012*, Tadayoshi Kohno (Ed.). USENIX Association, Bellevue, WA, USA, 205–220. https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/heninger

[34] RSA Security Inc. 1994. RSA Challenge List. http://www.ontko.com/pub/rayo/primes/rsa_fact.html accessed: 2025-04-14.

[35] Liz Izhikevich, Renata Teixeira, and Zakir Durumeric. 2021. LZR: Identifying Unexpected Internet Services. In *USENIX Security 2021*, Michael Bailey and Rachel Greenstadt (Eds.). USENIX Association, 3111–3128. https://www.usenix.org/conference/usenixsecurity21/presentation/izhikevich

[36] Mobin Javed and Vern Paxson. 2013. Detecting stealthy, distributed SSH bruteforcing. In *ACM CCS 2013*, Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung (Eds.). ACM Press, Berlin, Germany, 85–96. https://doi.org/10.1145/2508859.2516719

[37] Simon Josefsson and Ilari Liusvaara. 2017. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032. https://doi.org/10.17487/RFC8032

[38] Ioannis Koniaris, Georgios Papadimitriou, and Petros Nicopolitidis. 2013. Analysis and visualization of SSH attacks using honeypots. In *Eurocon 2013*. 65–72. https://doi.org/10.1109/EUROCON.2013.6624967

[39] Hendrik W Lenstra Jr. 1987. Factoring integers with elliptic curves. *Annals of mathematics* (1987), 649–673.

[40] Chris M. Lonvick and Sami Lehtinen. 2006. The Secure Shell (SSH) Protocol Assigned Numbers. RFC 4250. https://doi.org/10.17487/RFC4250

[41] Chris M. Lonvick and Tatu Ylonen. 2006. The Secure Shell (SSH) Authentication Protocol. RFC 4252. https://doi.org/10.17487/RFC4252

[42] Chris M. Lonvick and Tatu Ylonen. 2006. The Secure Shell (SSH) Connection Protocol. RFC 4254. https://doi.org/10.17487/RFC4254

[43] Chris M. Lonvick and Tatu Ylonen. 2006. The Secure Shell (SSH) Protocol Architecture. RFC 4251. https://doi.org/10.17487/RFC4251

[44] Chris M. Lonvick and Tatu Ylonen. 2006. The Secure Shell (SSH) Transport Layer Protocol. RFC 4253. https://doi.org/10.17487/RFC4253

[45] Solomon Z Melese and PS Avadhani. 2016. Honeypot system for attacks on SSH protocol. *International Journal of Computer Network and Information Security* 8, 9 (2016), 19.

[46] Damien Miller. 2024. *SSH Agent Protocol.* Internet-Draft draft-ietf-sshm-ssh-agent-01. Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-ietf-sshm-ssh-agent/01/ Work in Progress.

[47] Damien Miller and Markus Friedl. 2020. This document describes OpenSSH's support for U2F/FIDO security keys. https://cvsweb.openbsd.org/cgi-bin/cvsweb/src/usr.bin/ssh/PROTOCOL.u2f?rev=1.26 accessed: 2025-04-14.

[48] Matus Nemec, Marek Sys, Petr Svenda, Dusan Klinec, and Vashek Matyas. 2017. The Return of Coppersmith's Attack: Practical Factorization of Widely Used RSA Moduli. In *ACM CCS 2017*, Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM Press, Dallas, TX, USA, 1631–1648. https://doi.org/10.1145/3133956.3133969

[49] National Institute of Standards and Technology. 2013. *Digital Signature Standard (DSS).* Technical Report Federal Information Processing Standards Publications (FIPS PUBS) 186-4. U.S. Department of Commerce, Washington, D.C. https://doi.org/10.6028/NIST.FIPS.186-4

[50] National Institute of Standards and Technology. 2023. *Digital Signature Standard (DSS).* Technical Report Federal Information Processing Standards Publications (FIPS PUBS) 186-5. U.S. Department of Commerce, Washington, D.C. https://doi.org/10.6028/NIST.FIPS.186-5

[51] Jim Owens and Jeanna Matthews. 2008. A study of passwords and methods used in brute-force SSH attacks. In *USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET)*. 8.

[52] Jeonghoon Park, Jinsu Kim, Brij B Gupta, and Namje Park. 2021. Network log-based SSH brute-force attack detection model. *Computers, Materials & Continua* 68, 1 (2021).

[53] Kenneth G. Paterson and Gaven J. Watson. 2010. Plaintext-Dependent Decryption: A Formal Security Treatment of SSH-CTR. In *EUROCRYPT 2010 (LNCS, Vol. 6110)*, Henri Gilbert (Ed.). Springer Berlin Heidelberg, Germany, French Riviera, 345–361. https://doi.org/10.1007/978-3-642-13190-5_18

[54] Julien Piet, Aashish Sharma, Vern Paxson, and David A. Wagner. 2023. Network Detection of Interactive SSH Impostors Using Deep Learning. In *USENIX Security 2023*, Joseph A. Calandrino and Carmela Troncoso (Eds.). USENIX Association, Anaheim, CA, USA, 4283–4300. https://www.usenix.org/conference/usenixsecurity23/presentation/piet

[55] Damian Poddebniak, Juraj Somorovsky, Sebastian Schinzel, Manfred Lochter, and Paul Rösler. 2018. Attacking Deterministic Signature Schemes Using Fault Attacks. In *2018 IEEE European Symposium on Security and Privacy*. IEEE Computer Society Press, London, United Kingdom, 338–352. https://doi.org/10.1109/EuroSP.2018.00031

[56] John M Pollard. 1978. Monte Carlo methods for index computation (mod p). *Mathematics of computation* 32, 143 (1978), 918–924.

[57] Thomas Pornin. 2013. Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA). RFC 6979. https://doi.org/10.17487/RFC6979

[58] Niels Provos and Peter Honeyman. 2001. ScanSSH: Scanning the Internet for SSH Servers. In *15th Systems Administration Conference (LISA 2001)*. USENIX Association, San Diego, CA. https://www.usenix.org/conference/lisa-2001/scanssh-scanning-internet-ssh-servers

[59] Keegan Ryan, Kaiwen He, George Arnold Sullivan, and Nadia Heninger. 2023. Passive SSH Key Compromise via Lattices. In *ACM CCS 2023*, Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda (Eds.). ACM Press, Copenhagen, Denmark, 2886–2900. https://doi.org/10.1145/3576915.3616629

[60] Dawn Xiaodong Song, David A. Wagner, and Xuqing Tian. 2001. Timing Analysis of Keystrokes and Timing Attacks on SSH. In *USENIX Security 2001*, Dan S. Wallach (Ed.). USENIX Association, Washington, DC, USA. http://www.usenix.org/publications/library/proceedings/sec01/song.html

[61] The CADO-NFS Development Team. 2017. CADO-NFS, An Implementation of the Number Field Sieve Algorithm. http://cado-nfs.inria.fr/ Release 2.3.0, accessed: 2025-04-14.

[62] Filippo Valsorda. 2015. ssh who am i. https://words.filippo.io/ssh-whoami-filippo-io/ accessed: 2025-04-14.

[63] Yuming Wu, Phuong M Cao, Alexander Withers, Zbigniew T. Kalbarczyk, and Ravishankar Krishnan Iyer. 2020. Mining Threat Intelligence from Billion-scale SSH Brute-Force Attacks. *Proceedings 2020 Workshop on Decentralized IoT Systems and Security* (2020).

[64] Moosa Yahyazadeh, Sze Yiu Chau, Li Li, Man Hong Hue, Joyanta Debnath, Sheung Chiu Ip, Chun Ngai Li, Md. Endadul Hoque, and Omar Chowdhury. 2021. Morpheus: Bringing The (PKCS) One To Meet the Oracle. In *ACM CCS 2021*, Giovanni Vigna and Elaine Shi (Eds.). ACM Press, Virtual Event, Republic of Korea, 2474–2496. https://doi.org/10.1145/3460120.3485382

[65] Scott Yilek, Eric Rescorla, Hovav Shacham, Brandon Enright, and Stefan Savage. 2009. When private keys are public: results from the 2008 Debian OpenSSL vulnerability. In *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement* (Chicago, Illinois, USA) *(IMC '09)*. Association for Computing Machinery, New York, NY, USA, 15–27. https://doi.org/10.1145/1644893.1644896